```
MyClass.instance_eval do
  def class_method
    puts "In a class method"
 end
end

MyClass.class_method
```

produces:

```
In a class method
```

It might be helpful to remember that class_eval and instance_eval refer to the context in which self is being replaced—and not the kind of method that is defined inside the block. For method definition, class_eval defines instance methods, and instance_eval defines class methods.

Ruby has variants of these methods. instance_exec, class_exec, and module_exec behave identically to their _eval counterparts but only have the block version (that is, they do not take a string). Any arguments given to the methods are passed to the block as block parameters. This is an important feature. Using _eval, it is impossible to pass an instance variable into a block given to one of the _eval methods—because self is changed by the call, these variables go out of scope. With the _exec form, you can pass them in:

```
@animal = "cat"
"dog".instance_exec(@animal) do |other|
  puts "#{other} and #{self}"
end
```

produces:

```
cat and dog
```

instance_eval and Constants

When using instance_eval, constants are looked up in the lexical scope in which they were referenced. This (artificial) example shows the behavior:

```
module One
  CONST = "Defined in One"
  def self.eval_block(&block)
    instance_eval(&block)
  end
end

module Two
  CONST = "Defined in Two"
  def self.call_eval_block
    One.eval_block do
      CONST
    end
  end
end

Two.call_eval_block    # => "Defined in Two"
```

instance_eval and Domain-Specific Languages

It turns out that instance_eval has a pivotal role to play in a certain type of domain-specific language (DSL). For example, we might be writing a simple DSL for turtle graphics. In turtle